# CONDUCTOR CERCANO

Informe técnico · Asignación de conductores con Quadtree, Haversine, heap y Dijkstra propios sobre la red vial real de Lima Metropolitana y Callao · 2026-11-12

## 1. Introducción y planteamiento del problema

Una plataforma de movilidad debe responder, en milisegundos, a la pregunta: *entre miles de conductores en movimiento, ¿cuál puede llegar primero al pasajero?* La respuesta ingenua (medir la distancia a todos) escala linealmente y además usa la distancia en línea recta, que no refleja calles de sentido único ni la topología vial. CONDUCTOR CERCANO resuelve ambos problemas: un **Quadtree propio** reduce la búsqueda a una vecindad, **Haversine** y un **heap** seleccionan los k=5 candidatos globalmente más cercanos, y **Dijkstra propio** sobre el grafo vial dirigido de OpenStreetMap calcula la distancia real y el ETA de cada uno; se asigna el de menor ETA. Todo se ejecuta en un backend real con simulación de la flota sobre las calles de 46 distritos.

## 2. Arquitectura del sistema

**Backend** (Python, FastAPI, Uvicorn): capas separadas en *algoritmos/* (Haversine, Dijkstra), *estructuras/* (NodoQuadtree, Quadtree, GrafoVial), *simulacion/* (Conductor, Pasajero, Flota, Despachador, MotorSimulacion, sin dependencias web) y *backend/* (API REST, WebSocket, servicios, SQLAlchemy/SQLite). **Datos**: los límites de distritos se extraen de las relaciones administrativas de OSM; la red vial se obtiene con OSMnx (*network_type=drive*, grafo dirigido con sentidos de circulación) desde Overpass o, si no responde, desde un extracto PBF de OSM recortado al polígono del distrito; se guarda en *datos/grafos/* (GraphML) y no se vuelve a descargar. **Tiempo real**: en cada turno el servidor consulta el Quadtree con la vista de cada cliente y envía solo los conductores que cambiaron (deltas cuantizados a ~1 m), con control de frecuencia por cliente y reconexión automática. **Frontend** (React, TypeScript, Vite, Tailwind, Framer Motion, Zustand, MapLibre GL y Three.js): dashboard, centro de operaciones 2D/3D (vehículos en InstancedMesh orientados por rumbo, edificios extruidos, rutas con profundidad), solicitud de servicio con geocodificación Nominatim, historial y configuración. Persistencia de solicitudes, asignaciones, rutas, configuraciones, semillas, resultados y métricas; las posiciones por turno no se guardan.

## 3. Diseño e implementación del Quadtree

Cada **NodoQuadtree** guarda sus límites, sus elementos (diccionario id→entrada), la capacidad, la profundidad, su padre, un contador del subárbol y referencias a cuatro hijos (SO, SE, NO, NE). Una hoja se subdivide al superar la capacidad (16), salvo que alcance la profundidad máxima (20) o que todos sus elementos compartan coordenadas, lo que evita subdivisiones infinitas. Un índice auxiliar id→hoja permite eliminar en O(profundidad) y garantiza que no haya duplicados; al eliminar se fusionan los subárboles que vuelven a caber en un nodo. **actualizar_posicion** comprueba en O(1) si el punto sigue en la misma hoja (región semiabierta [mín, máx)); si no, elimina y reinserta. **k vecinos exactos**: se consulta el rectángulo R(r) que contiene al círculo de radio r ($\Delta\phi = \delta$, $\Delta\lambda = \text{asin}(\sin\delta / \cos\phi)$, exacto en la esfera), empezando en r = 250 m y duplicando r. Todo punto fuera de R(r) está a más de r: es la *cota inferior de las regiones no exploradas*. Cuando hay k candidatos disponibles a distancia ≤ r, ningún punto no explorado puede superarlos y el resultado es globalmente óptimo; los k mejores se eligen con *heapq.nsmallest*. El mismo Quadtree indexa los nodos del grafo para ubicar direcciones en la red vial (sin índices espaciales de OSMnx).

## 4. Implementación de Haversine y Dijkstra

**Haversine**: $d = 2R \cdot \text{asin}(\sqrt{\sin^2(\Delta\phi/2) + \cos\phi_1 \cos\phi_2 \sin^2(\Delta\lambda/2)})$ con R = 6 371 008,8 m (radio medio IUGG), validación de rangos y recorte numérico; se prueba con puntos idénticos, 0,001° (111,195 m), Plaza Mayor–Parque Kennedy (≈8,4 km), antípodas y antimeridiano. **Dijkstra**: lista de adyacencia dirigida (se conserva la arista paralela más corta, longitudes en metros de OSM), cola de prioridad con heapq y *lazy deletion*, predecesores para reconstruir la ruta, parada temprana al extraer el destino y manejo explícito de nodos inexistentes y de destinos inalcanzables. El ETA de recogida es (resto de la arista actual + distancia Dijkstra desde el siguiente nodo) / velocidad en m/s; la velocidad media configurable es 30 km/h con variación individual ±15 %. La ruta del viaje se calcula después de elegir al conductor.

## 5. Integración de las estructuras de datos

El **Despachador** ejecuta, bajo un cerrojo compartido con el motor (sin doble asignación): validación → ubicación de origen y destino en nodos circulables (Quadtree de nodos, máx. 400 m; si no hay red se informa «no existe una ruta válida») → k=5 disponibles más cercanos (Quadtree + Haversine + heap) → Dijkstra por candidato (los que no tienen ruta se descartan sin detener el proceso; si ninguno sirve se evalúa un nuevo lote) → menor ETA → cambio de estado → registro y notificación por WebSocket con una justificación textual. El **motor** avanza a cada conductor por aristas válidas del grafo dirigido (sin giros en U salvo en calles ciegas), actualiza el Quadtree tras cada movimiento y gestiona el ciclo DISPONIBLE → OCUPADO → EN_SERVICIO → DISPONIBLE. Tres generadores derivados de una semilla (flota, movimiento y demanda) hacen la simulación reproducible.

## 6. Análisis Big-O

| Operación | Complejidad | Comentario |
|---|---|---|
| Haversine | O(1) | Operaciones trigonométricas constantes |
| Quadtree: insertar / eliminar | O(h), h ≤ 20 | $h \approx \log_4(n/c)$ con distribución urbana; fusión amortizada |
| Quadtree: actualizar posición | O(1) misma hoja; O(h) si cambia | La mayoría de movimientos de 2 s no cambia de hoja |
| Quadtree: rectángulo | O(h + m + nodos visitados) | m = elementos en las hojas intersecadas |
| k vecinos (expansión de radio) | O(Σ iteraciones · (h + m)) + O(m log k) | La serie geométrica acota el total a O(última consulta) |
| Búsqueda lineal | O(n log k) | Examina siempre los n conductores |
| Dijkstra (heap binario) | O((V + E) log V) | Parada temprana: explora una fracción del grafo |
| Turno de simulación | O(n · h) | Avance por aristas + actualización del índice |
| Asignación completa | O(h + m + k (V + E) log V) | Dominada por las k ejecuciones de Dijkstra |

## 7. Resultados de los benchmarks

Mediciones reales en Windows 11, Python 3.14.8, 16 núcleos lógicos, 13.8 GB. Búsqueda: distrito san-juan-de-lurigancho (14 336 nodos), conductores ubicados sobre calles reales, 30 % ocupados, 200 consultas × 5 repeticiones, k = 5, mismos datos para ambas estructuras. Exactitud = coincidencia de los k identificadores con la búsqueda exhaustiva.

| Conductores | Lineal (ms) | Quadtree (ms) | σ QT (ms) | Examinados lineal / QT | Aceleración | Exactitud | Act. QT (µs/cond.) |
|---|---|---|---|---|---|---|---|
| 300 | 0.629 | 0.1461 | 0.2025 | 300 / 81.6 | ×4.3 | 200/200 | 9.9 |
| 3 000 | 6.002 | 0.1676 | 0.1121 | 3 000 / 59.0 | ×35.8 | 200/200 | 12.5 |
| 30 000 | 63.901 | 0.5561 | 0.3502 | 30 000 / 161.0 | ×114.9 | 200/200 | 14.8 |

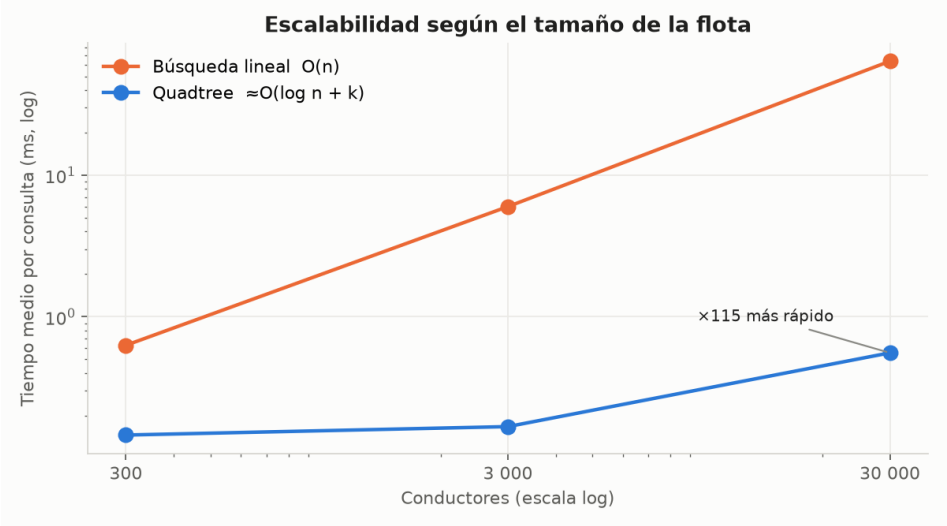



Figura 1. Tiempo medio por consulta según el tamaño de la flota (escala log-log).

Con 30 000 conductores el Quadtree responde en 0.556 ms frente a 63.9 ms de la búsqueda lineal (×115) examinando 161 elementos en vez de 30 000. Con 300 conductores la ventaja es menor porque la flota es dispersa y el radio debe expandirse varias veces. La memoria del índice para 30 000 conductores es ≈6.1 MB (6225 nodos, profundidad 9).

**Dijkstra**: 100 pares aleatorios de nodos por distrito sobre el mismo DiGraph. NetworkX se usa exclusivamente como referencia: 600/600 distancias coinciden (tolerancia 1e-06 m).

| Distrito | Nodos | Aristas | Propio (ms) | NetworkX (ms) | Nodos explorados | Coinciden | Dif. máx. (m) |
|---|---|---|---|---|---|---|---|
| miraflores | 1 681 | 3 219 | 1.87 | 2.73 | 806 (48 %) | 100/100 | 0.0e+00 |
| san-isidro | 1 696 | 3 244 | 2.46 | 3.56 | 880 (52 %) | 100/100 | 0.0e+00 |
| lima | 3 124 | 6 372 | 4.10 | 7.96 | 1483 (47 %) | 100/100 | 0.0e+00 |
| santiago-de-surco | 4 463 | 10 110 | 6.47 | 11.90 | 2062 (46 %) | 100/100 | 0.0e+00 |
| callao | 6 005 | 15 451 | 11.14 | 17.06 | 3260 (54 %) | 100/100 | 0.0e+00 |
| san-juan-de-lurigancho | 14 336 | 37 927 | 20.93 | 39.53 | 7047 (49 %) | 100/100 | 0.0e+00 |

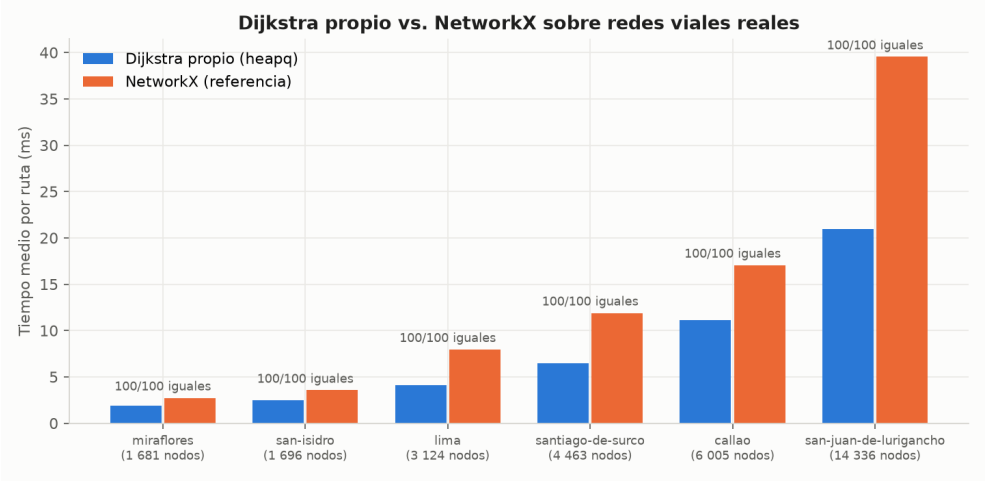



Figura 2. Tiempo medio por ruta: Dijkstra propio frente a NetworkX en seis distritos reales.

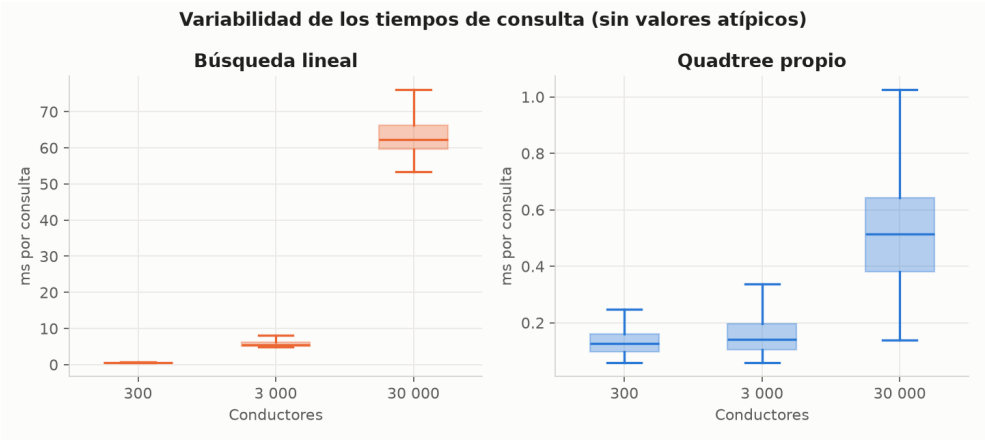



Figura 3. Variabilidad de los tiempos de consulta por método y tamaño de flota.

## 8. Comparación con H3 de Uber

| Criterio | Búsqueda lineal | Quadtree (este proyecto) | H3 (Uber) |
|---|---|---|---|
| Organización espacial | Ninguna (lista) | Árbol adaptativo: subdivide solo donde hay densidad | Malla jerárquica fija de hexágonos (16 resoluciones) |
| Complejidad de consulta | O(n) | O(h + m) por rectángulo; k-NN exacto con cota inferior | O(1) por celda; k-ring de celdas vecinas $O(k^2)$ |
| Actualización de posición | O(1) | O(1) en la misma hoja; O(h) si cambia | O(1): recalcular índice de celda y mover entre cubetas |
| Consultas por cercanía | Exactas pero costosas | Exactas (radio creciente + cota inferior) | Aproximadas por anillos; exactitud requiere verificar distancias |
| Ventajas | Simple, sin estructura | Se adapta a densidades muy desiguales (centro vs. periferia) | Vecinos equidistantes, agregación analítica, celdas estables |
| Desventajas | No escala | Rebalanceo con movimientos; rectángulos distorsionados en latitudes altas | Resolución fija: celdas vacías o saturadas; dependencia externa |
| Escalabilidad | Mala (lineal) | Buena en memoria de un proceso | Muy buena y distribuible (claves de celda en cachés/particiones) |

H3 es superior para sistemas distribuidos a escala de ciudad entera (las claves de celda se reparten entre servidores y sirven para análisis de oferta y demanda), mientras que el Quadtree brilla dentro de un proceso con distribuciones heterogéneas y garantiza resultados exactos. H3 no se utiliza en la implementación: es solo un punto de comparación conceptual.

## 9. Conclusiones

(1) El Quadtree propio logra búsquedas exactas de los 5 conductores disponibles más cercanos hasta ×115 más rápidas que la búsqueda lineal con 30 000 conductores, con 600/600 resultados idénticos a la fuerza bruta. (2) El Dijkstra propio reproduce las distancias de NetworkX en 600/600 casos y, gracias a la parada temprana y a la lista de adyacencia compacta, es más rápido en todos los distritos medidos. (3) La distancia en línea recta no basta: en los mapas generados el candidato más cercano en Haversine a menudo no es el de menor ETA por los sentidos de circulación, lo que justifica evaluar rutas reales. (4) La arquitectura separa dominio, servicios y presentación, persiste el historial en SQLite, transmite

deltas por WebSocket y es reproducible por semilla. Como trabajo futuro: Dijkstra multiobjetivo inverso (una sola ejecución para los k candidatos), A* con heurística Haversine y velocidades por tipo de vía.